

Last time: Collision resistant hash function

Experiment $\text{col}_{A,\Pi}(n)$:

1. A key s is generated by $\text{Gen}(1^n)$
2. The adversary A is given s , and outputs x, x' ($\in \{0,1\}^{l'}$ if fixed-length)
3. If $H^s(x) = H^s(x')$ and $x \neq x'$, A has found a collision and the experiment outputs 1

A hash function (Gen, H) is collision resistant if for all PPT A , there is a negl such that

$$\Pr[\text{col}_{A,\Pi}(n)=1] \leq \text{negl}(n)$$

One-way function

- Sufficient and necessary for almost all symmetric-key primitives
- One-wayness does not imply collision resistance
- Collision resistance does not imply one-wayness

Some properties of hash function

- “ratio of shrinking”

$$H: \{0,1\}^{512} \rightarrow \{0,1\}^{256}$$

$$H: \{0,1\}^{1024} \rightarrow \{0,1\}^{256}$$

- Hardness of finding a collision

- $H: \{0,1\}^{512} \rightarrow \{0,1\}^{256}$

- $H: \{0,1\}^{1024} \rightarrow \{0,1\}^{256}$

- $H: \{0,1\}^{2048} \rightarrow \{0,1\}^{256}$

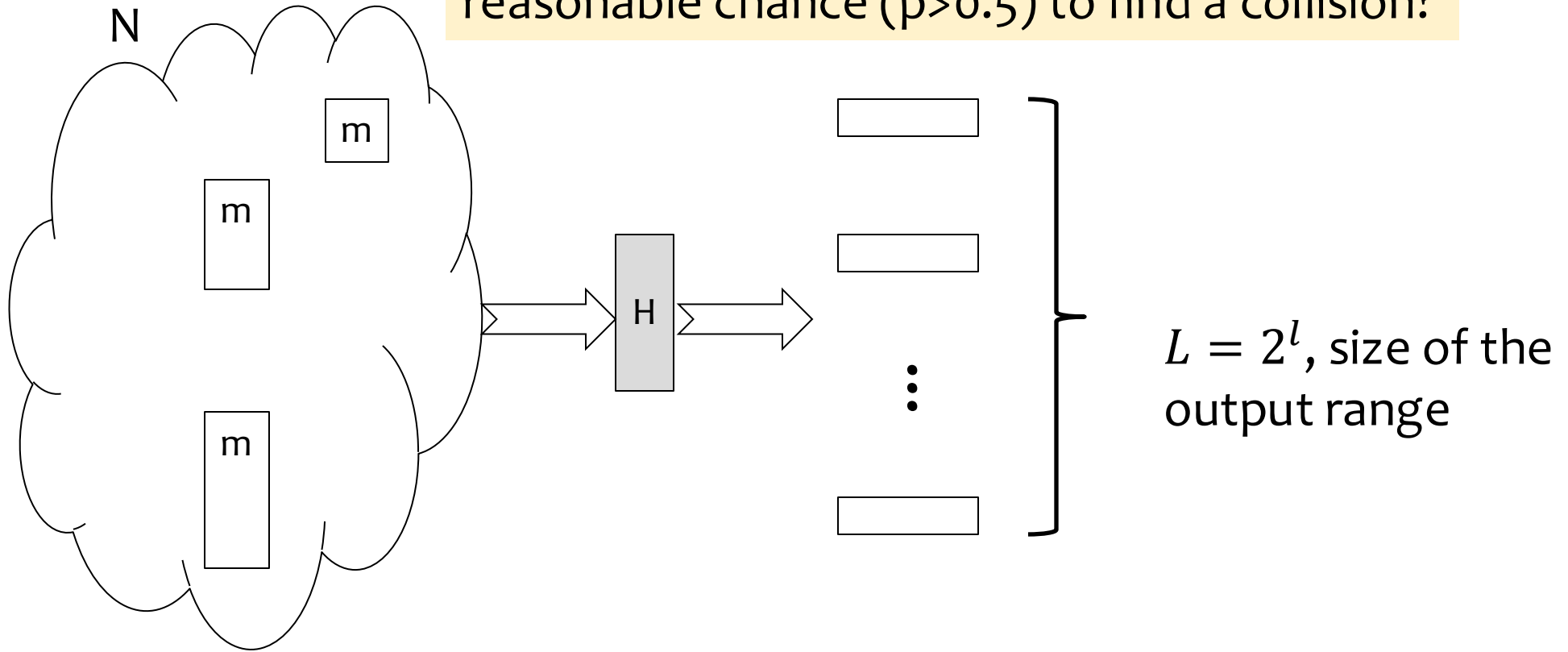
Ratio of collision: $\frac{2^{l-256}}{2^l} = \frac{1}{2^{256}}$

Think about Hash table

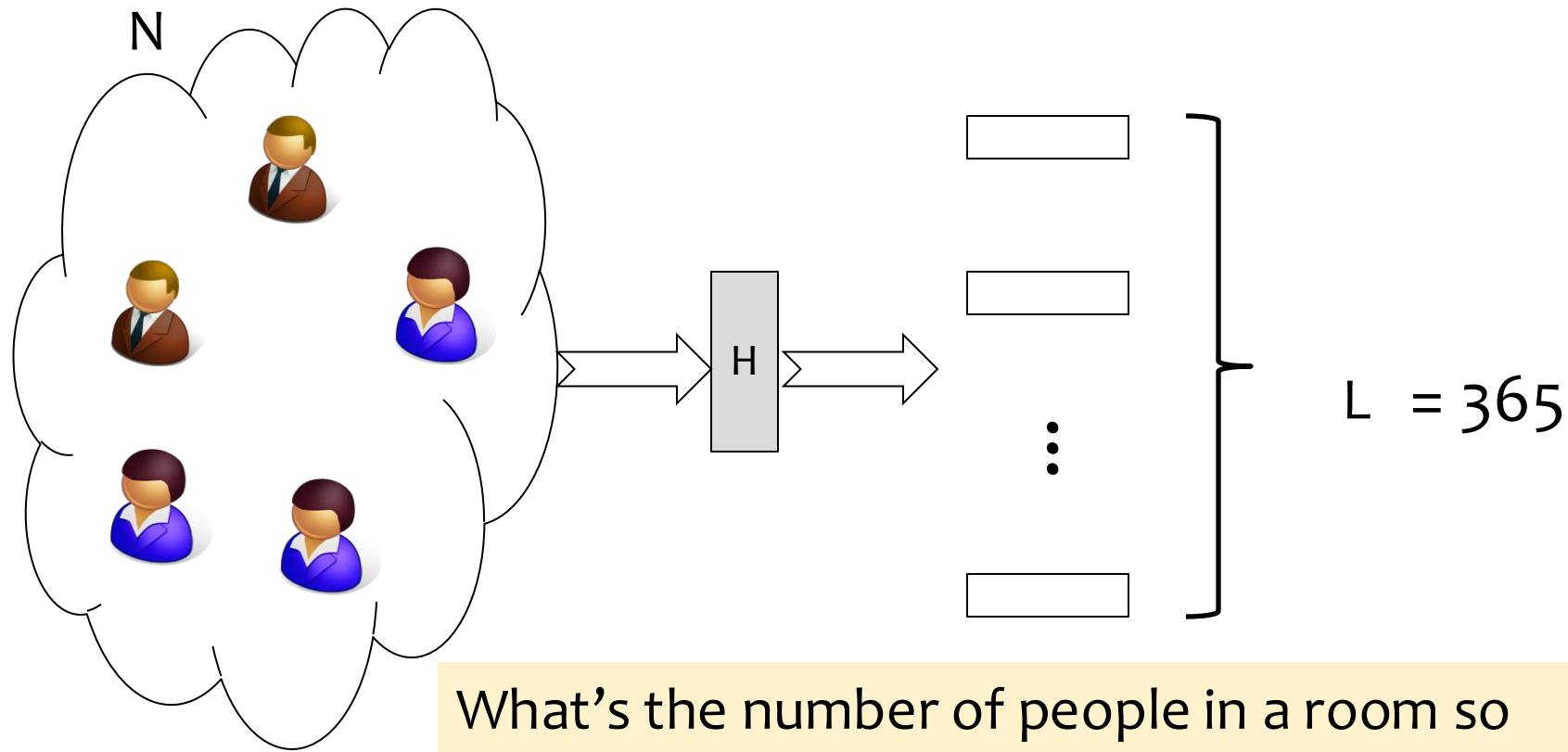
Hardness of a secure hash only depends on the **output length**, but not the input length

How hard is it to find collision?

How large does N has to be to have a reasonable chance ($p > 0.5$) to find a collision?



Birthday attack

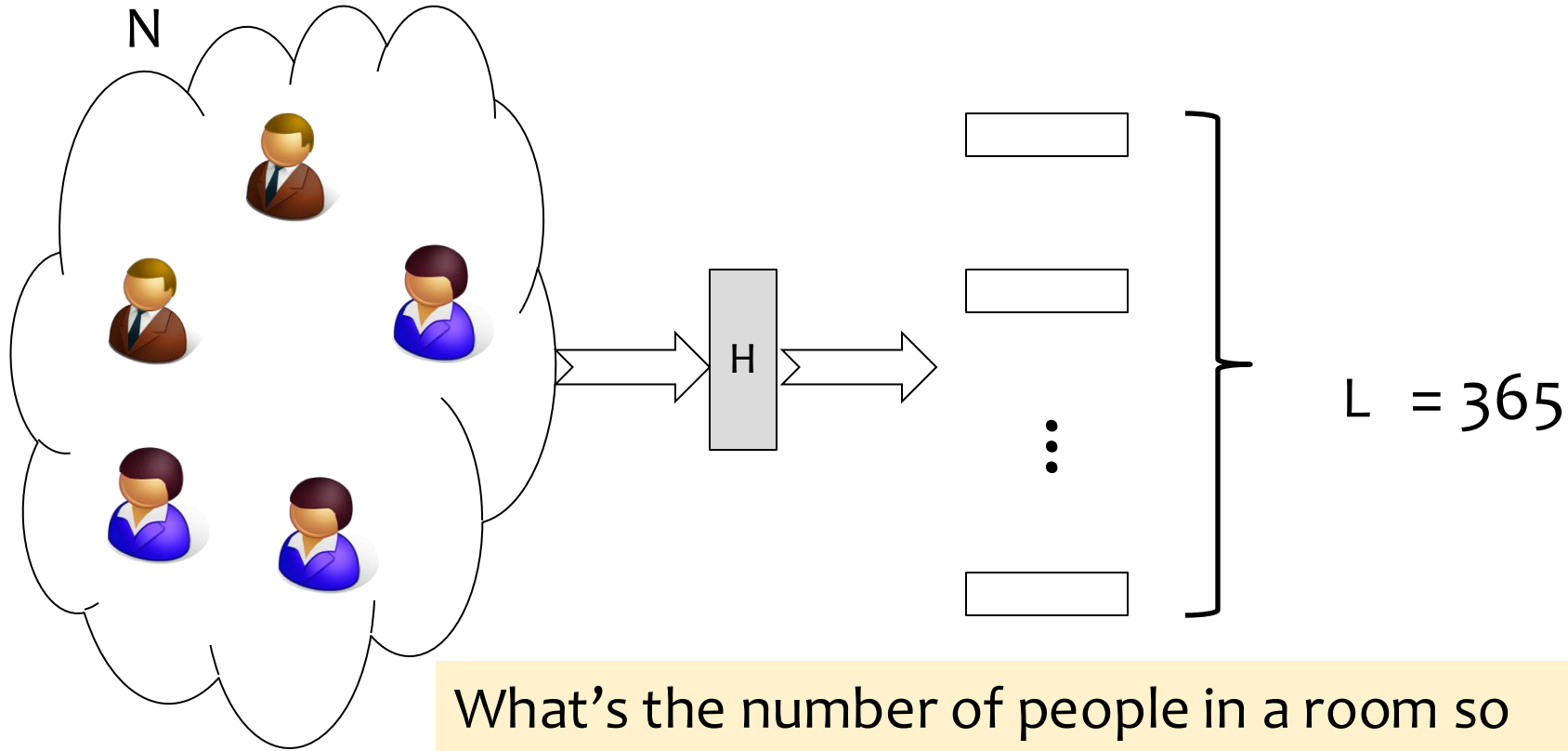


What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that two people in that room share the same birthday?

Birthday attack

$N=23$ $p=0.5$

$N=60$ $p = 0.99$



What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that two people in that room share the same birthday?

Birthday attack: proof

- Probability that all birthdays are different

$$\overline{p(N)} = 1 \times \frac{364}{365} \times \frac{363}{365} \times \dots \frac{365-N+1}{365}$$

- Probability that at least two birthdays are identical

$$p(N) = 1 - 1 \times \frac{364}{365} \times \frac{363}{365} \times \dots \frac{365-N+1}{365}$$

- Taylor approximation: $e^{-x} \approx 1 - x \Rightarrow 1 - \frac{1}{365} \approx e^{-\frac{1}{365}}$

- So: $p(N) \approx 1 - \left(e^{-\frac{1}{365}} \cdot e^{-\frac{2}{365}} \cdot \dots \cdot e^{-\frac{N-1}{365}} \right) = 1 - e^{-\frac{N(N-1)}{2 \times 365}} \approx 1 - e^{\frac{-N^2}{2 \times 365}}$

How many people needed for a collision?

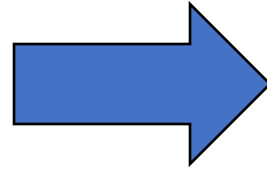
- Greater than p iff

$$1 - e^{-\frac{N^2}{2 \times 365}} > p$$

$$\Rightarrow e^{-\frac{N^2}{2 \times 365}} < 1 - p$$

$$\Rightarrow \frac{N^2}{2 \times 365} > -\ln(1 - p)$$

$$\Rightarrow N > \sqrt{2 \times 365 \ln \frac{1}{1 - p}}$$

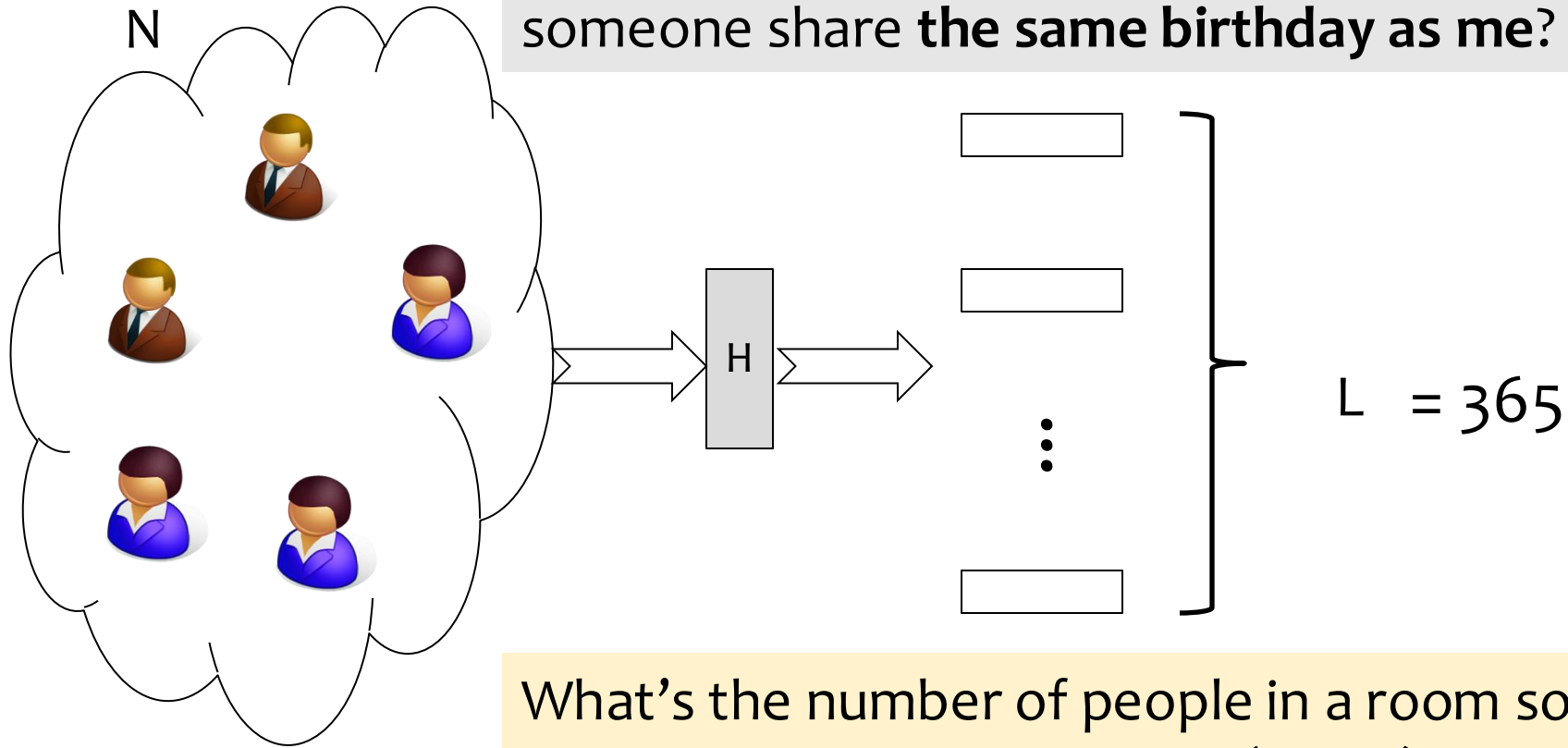


$$N > 1.17\sqrt{365} \approx 23$$

for $p = 0.5$

Why it is a "paradox"

What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that someone share **the same birthday as me?**

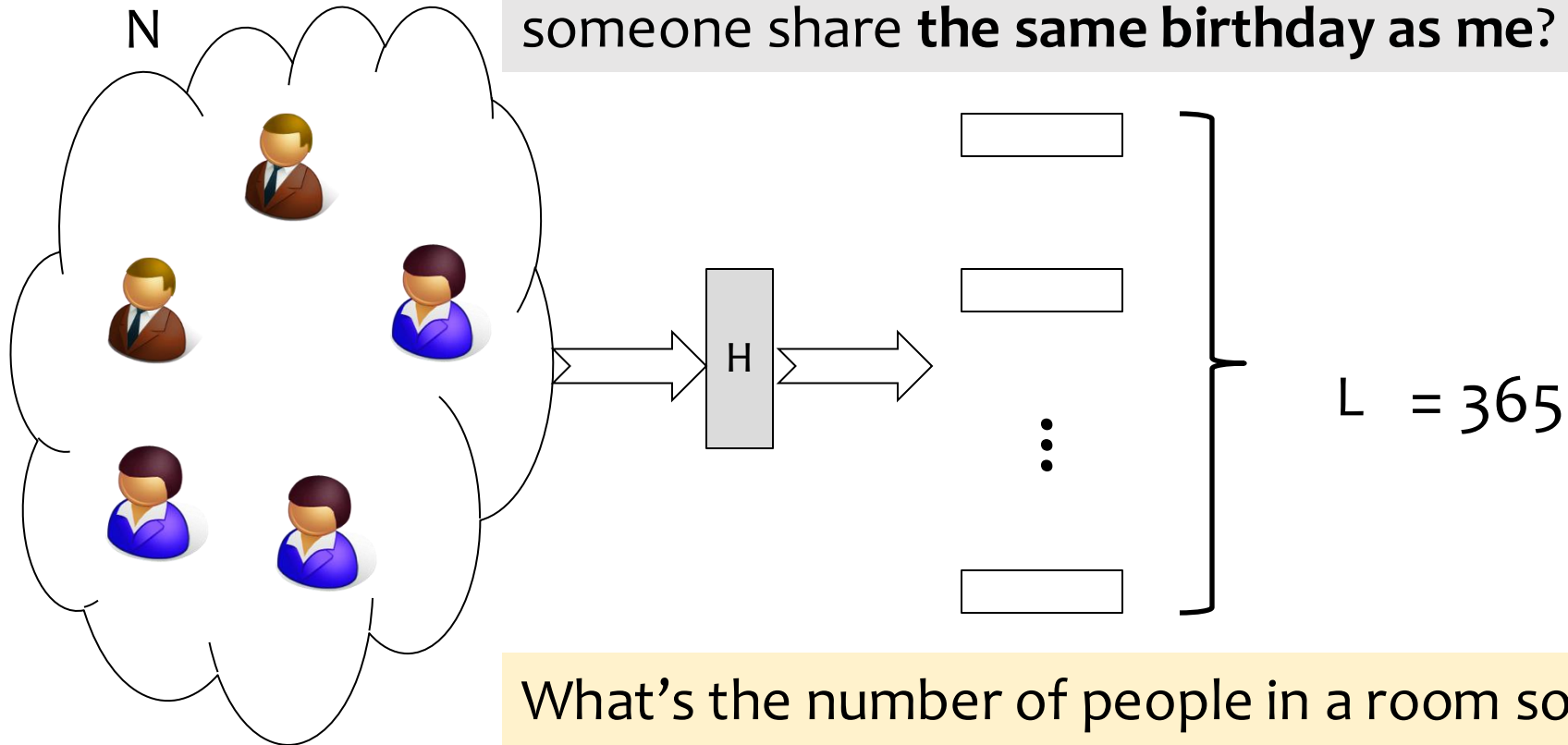


What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that two people in that room share **the same birthday?**

Why it is a paradox

Weak collision

What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that someone share **the same birthday as me?**

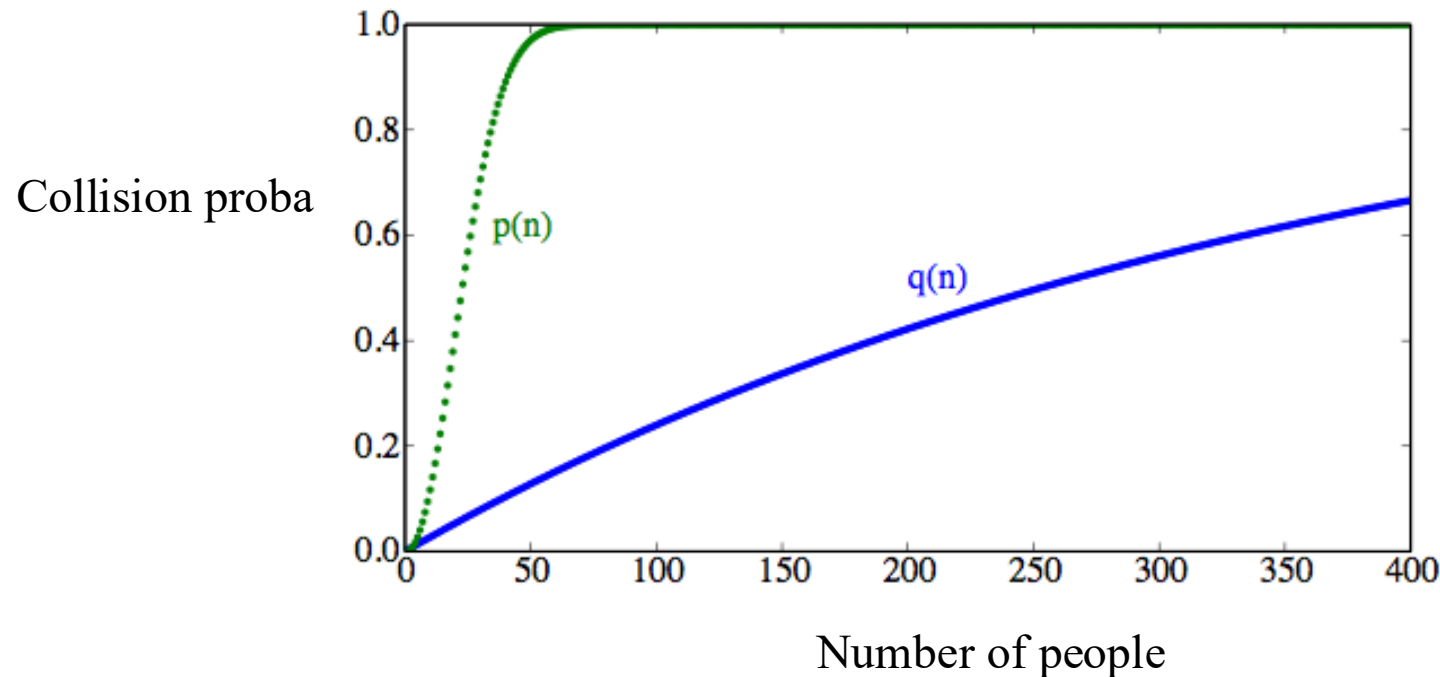


Strong collision

What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that two people in that room share **the same birthday?**

Why it is a paradox

- Two same birthdays: $p(n) \approx 1 - e^{-\frac{n^2}{2 \times 365}}$
- Same birthday as me: $q(n) = 1 - \left(\frac{365-1}{365}\right)^n$



Collision resistance

- Weak collision resistant: given x , infeasible to find x' such that $H(x') = H(x)$
- Strong collision resistant: infeasible to find any x, x' such that $H(x') = H(x)$

Birthday paradox: consequences

- For any hash function mapping a number N of inputs to a number $L = 2^l$ of outputs, there is a pretty good probability of finding a collision if one can try a number of $N > \sqrt{L}$ inputs
- For the hash function to be secure
 - it must be computationally infeasible to try a number of inputs in $O(\sqrt{L})$
 - Make sure the length of the hash is large enough!

Message-Digest Algorithm 5 (MD5) -- Broken

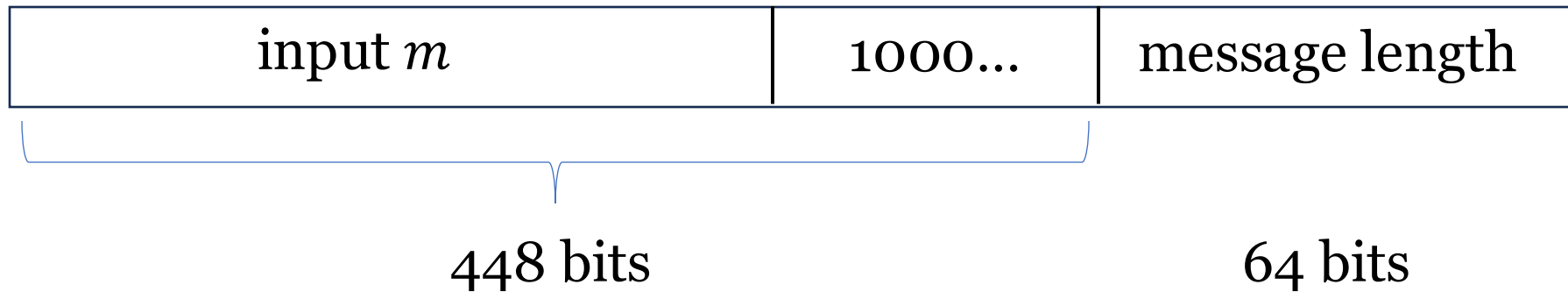
- Developed by Ron Rivest in 1991
- Uses 128-bit hash values
- How to Break MD5 and Other Hash Functions, Xiaoyun Wang and Hongbo Yu, Eurocrypt 2005 best paper award

Secure Hash Algorithm (SHA)

- Developed by NSA and approved as a federal standard by NIST
- **SHA-1** (1993)
 - 160-bits
 - Considered **insecure**
 - Still found in legacy applications
 - Vulnerabilities less severe than those of MD5
- **SHA-2 family** (2002)
 - 256 bits (SHA-256), 512 bits (SHA-512), etc.
- **SHA-3** (2015)

SHA 256

- 512-bit input to 256-bit output
- Input padding:



SHA 256

- extend



512 = 16 blocks, 32 bits each



64 blocks, 32 bits each

Extend the first 16 words into the remaining 48 words $w[16..63]$ of the message schedule array:

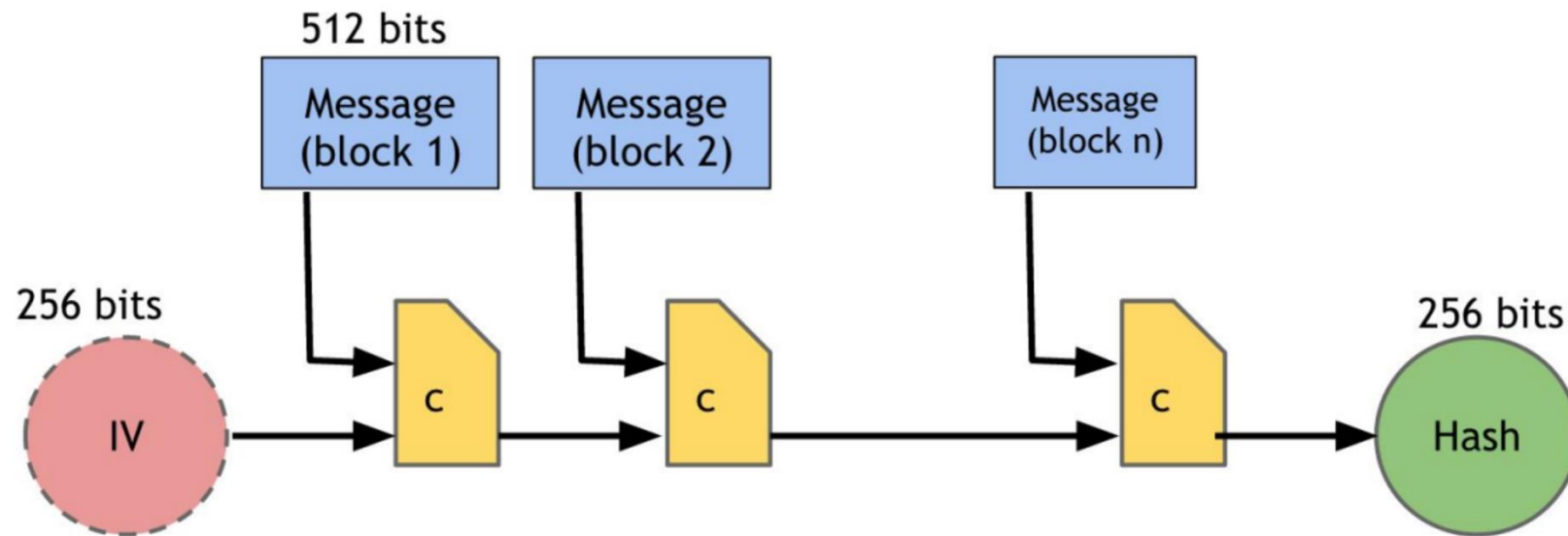
for i **from** 16 **to** 63

$s0 := (w[i-15] \text{ rightrotate } 7) \text{ xor } (w[i-15] \text{ rightrotate } 18) \text{ xor } (w[i-15] \text{ rightshift } 3)$

$s1 := (w[i-2] \text{ rightrotate } 17) \text{ xor } (w[i-2] \text{ rightrotate } 19) \text{ xor } (w[i-2] \text{ rightshift } 10)$

$w[i] := w[i-16] + s0 + w[i-7] + s1$

SHA 256

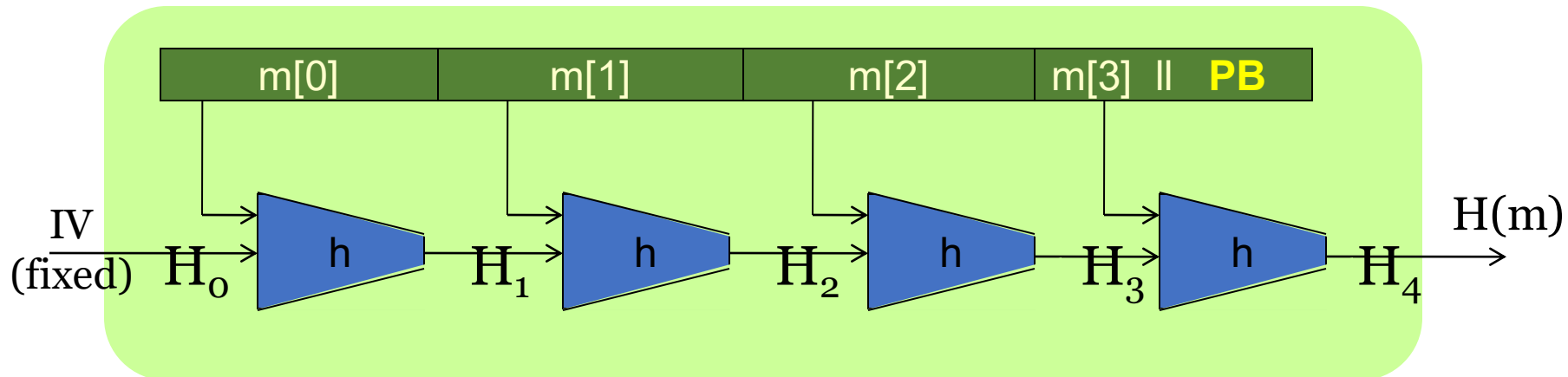


SHA 256 round function

- Initialize hash values: (the first 32 bits of the fractional parts of the square roots of the first 8 prime numbers)
 - $h_0 := \text{0x6a09e667}$
 - $h_1 := \text{0xbb67ae85}$
 - $h_2 := \text{0x3c6ef372}$
 - $h_3 := \text{0xa54ff53a}$
 - $h_4 := \text{0x510e527f}$
 - $h_5 := \text{0x9b05688c}$
 - $h_6 := \text{0x1f83d9ab}$
 - $h_7 := \text{0x5be0cd19}$
-
- $a := h_0, b := h_1, c := h_2, d := h_3, e := h_4, f := h_5, g := h_6, h := h_7$

Merkle-Damgard Transformation

Dealing with arbitrary length input



PB means the padding bits

Merkle-Damgard Transform

Construction: (Gen, h) a collision resistant hash function $\{0,1\}^{n+n'} \rightarrow \{0,1\}^n$

- Gen: unchanged
- H : for input string $x \in \{0,1\}^*$ of length $L < 2^{n'}$
 1. Padding: append 10000... concatenated with $L = |x|$ such that the result is a multiple of n'
 2. $y_0 = IV = 0^n$
 3. For $i = 1, 2, \dots, t$, $y_i = h(y_{i-1} || x_i)$

Merkle-Damgard Transform

Theorem: If h is collision resistant, then so is H .

Proof: collision on $H \Rightarrow$ collision on h

Suppose $H(M) = H(M')$

We build collision for h .

$$IV = H_0, H_1, \dots, H_{t-1}, H_t = H(M)$$

$$IV = H_0, H_1', \dots, H_{t'-1}', H_{t'}' = H(M')$$

Case 1: $|M| \neq |M'|$, padding is different!

$$h(H_{t-1}, M_t || PB) = H_t = H_{t'}' = h(H_{t'-1}', M_{t'}' || PB')$$

PB means the padding bits, M_t is the t -th block of M and $M_{t'}'$ is the t' -th block of M'

Merkle-Damgard Transform

Theorem: If h is collision resistant, then so is H .

Proof: collision on $H \Rightarrow$ collision on h

Suppose $H(M) = H(M')$

We build collision for h .

$$IV = H_0, H_1, \dots, H_{t-1}, H_t = H(M)$$

$$IV = H_0, H_1', \dots, H_{t'-1}', H_{t'}' = H(M')$$

Case 2: $|M| = |M'|$, then $t = t'$ and the padding is the same

$$h(H_{t-1}, M_t || PB) = H_t = H_t' = h(H_{t-1}', M_t' || PB)$$

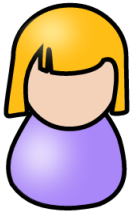
$H_{t-1} = H_{t-1}'$ or not?

Merkle Hash tree

Merkle Hash tree

Integrity of Storage

client



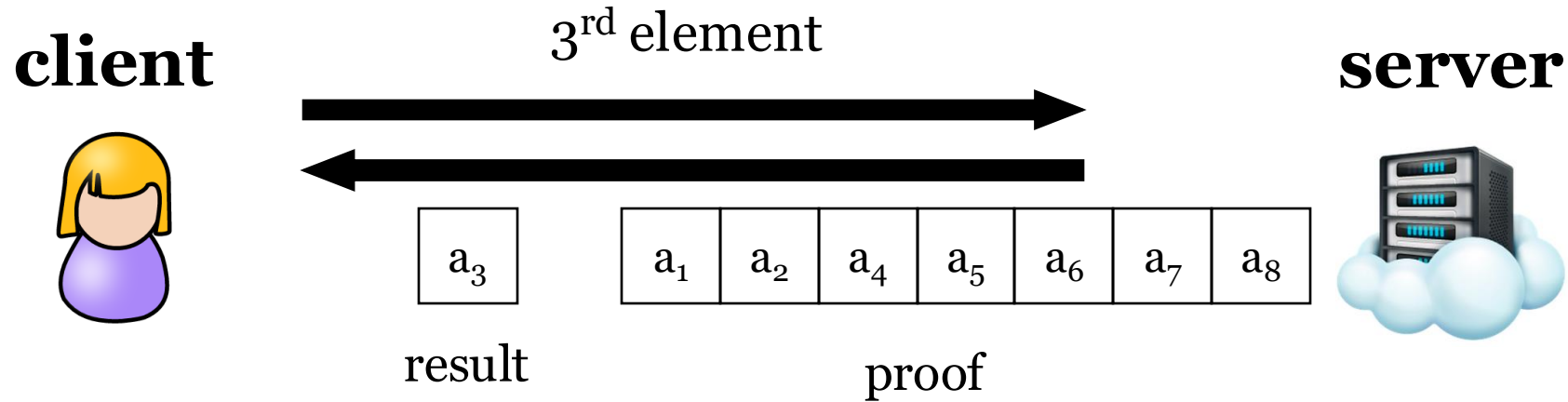
$H(\text{📄})$



server



Integrity of storage



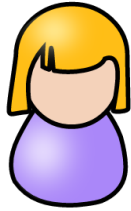
$$\text{digest} = H(a_1, a_2, \dots, a_8)$$

a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
-------	-------	-------	-------	-------	-------	-------	-------

Verification: $\text{digest} = H(a_1, a_2, \dots, a_8)$

Merkle hash tree

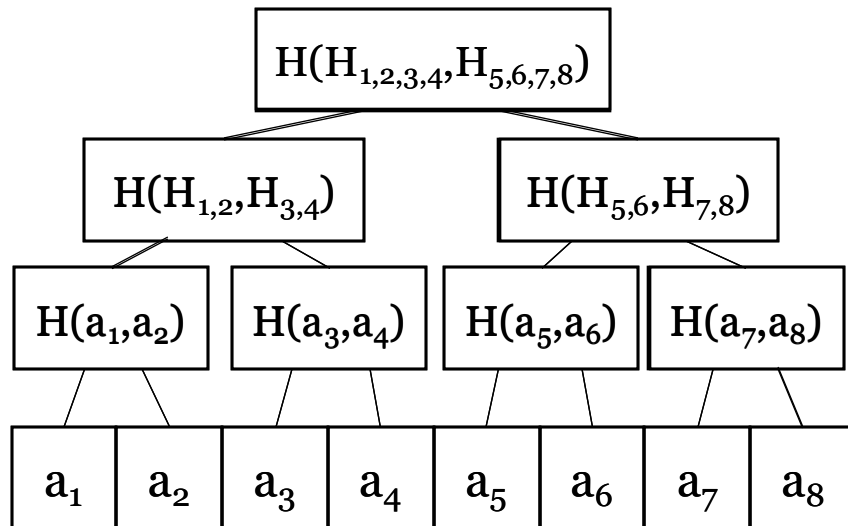
client



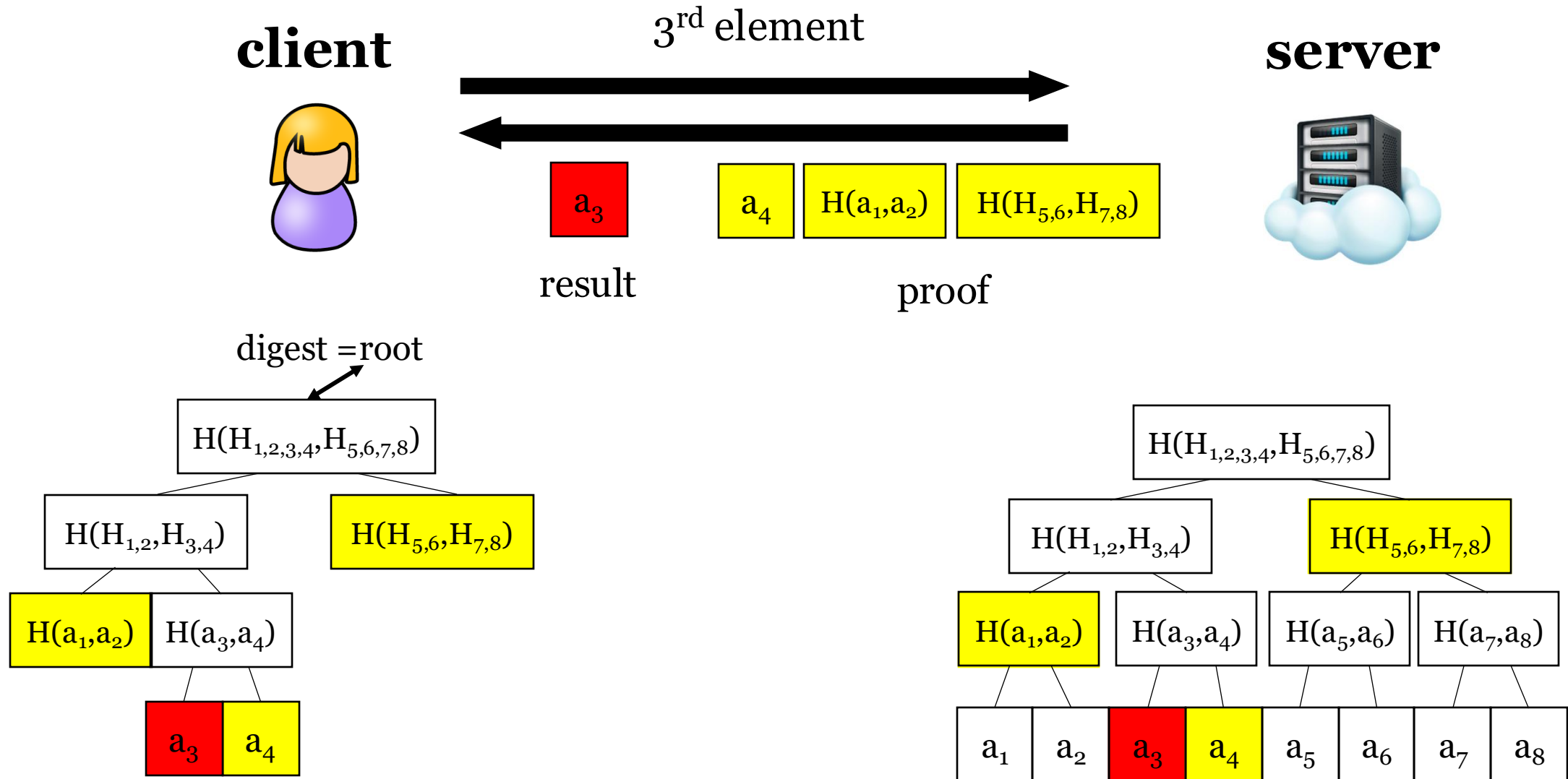
server



digest = root

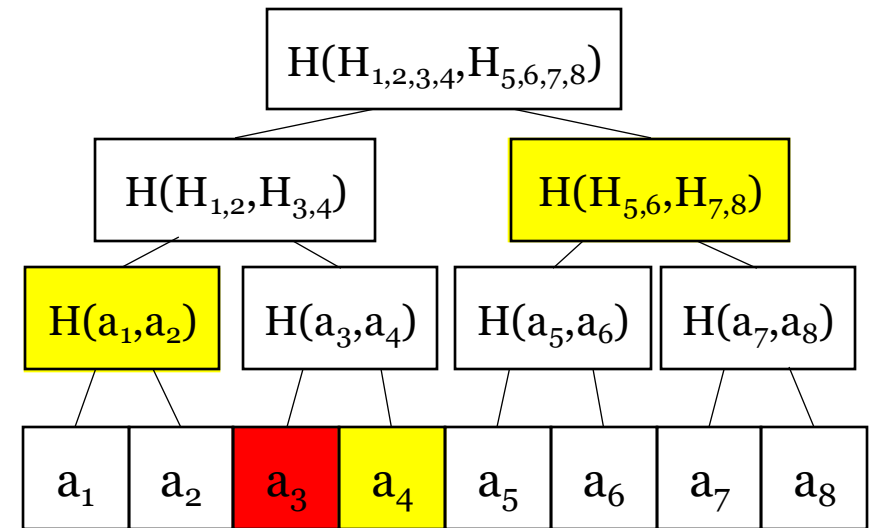
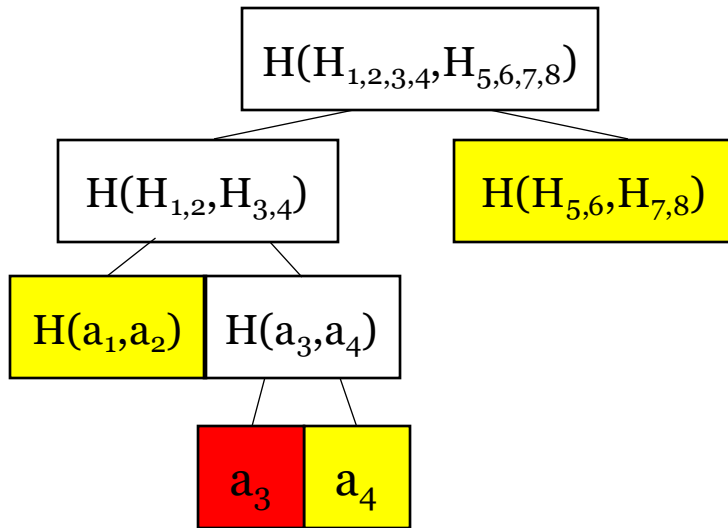


Merkle hash tree



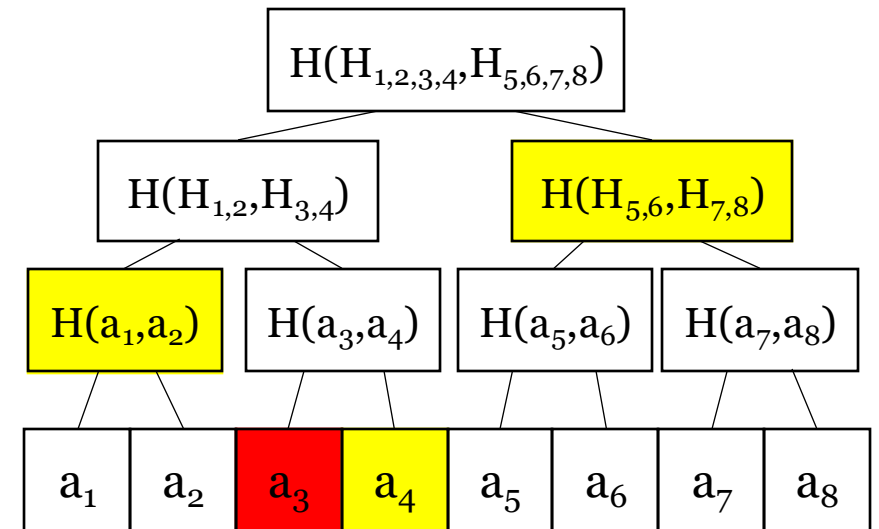
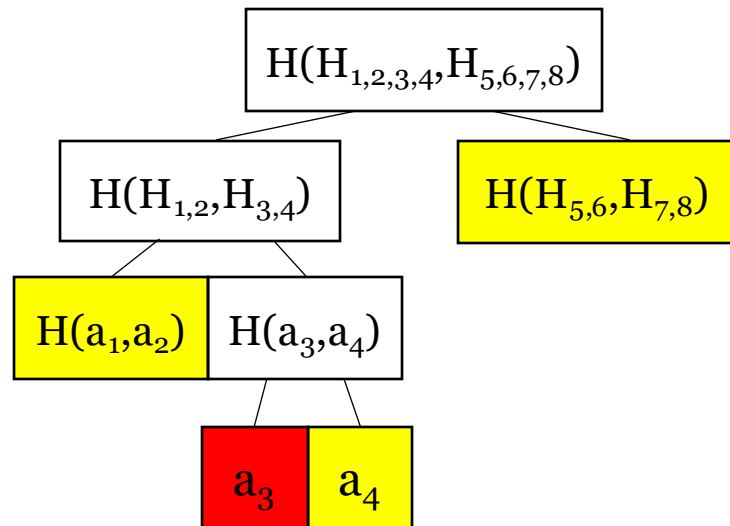
Merkle hash tree

- Correctness
- Soundness: cheat $\rightarrow$ break collision resistant of hash function

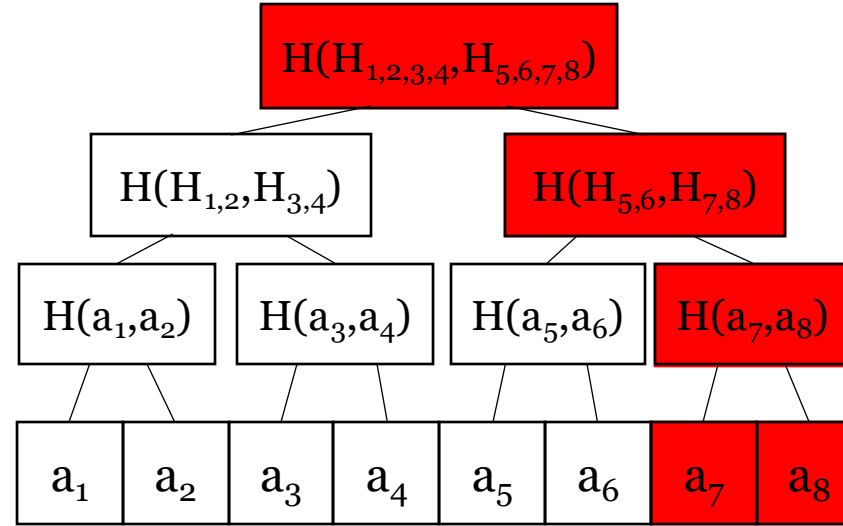
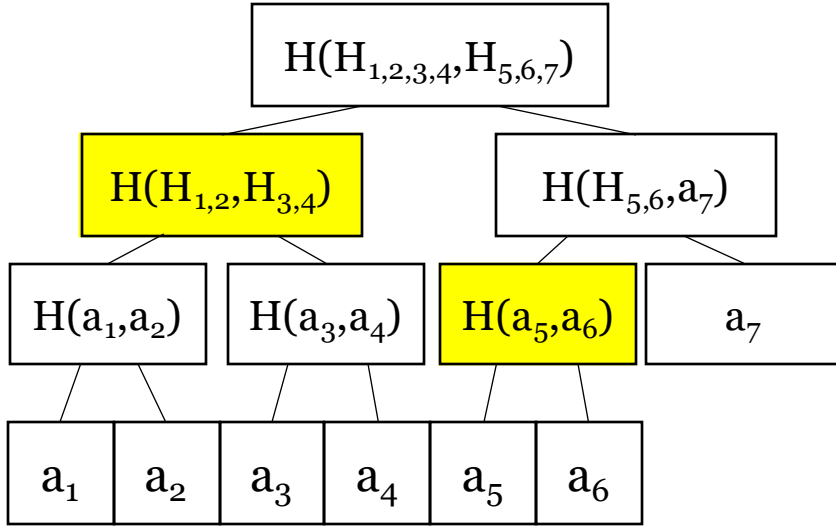


Complexity

- Local storage: $O(1)$
- Server time: $O(\log n)$ with $O(n)$ storage
- Communication: $O(\log n)$
- Verification time: $O(\log n)$



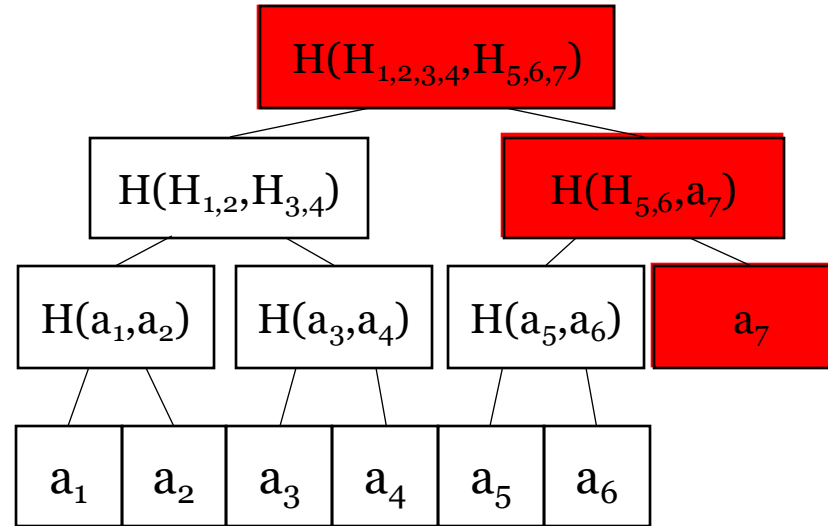
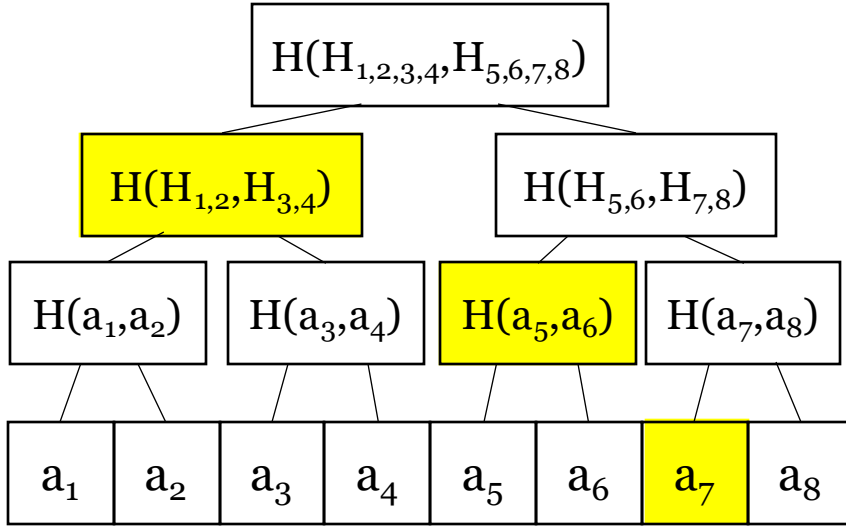
Update



Insertion:

1. P sends proof for a_7 , V validates it
2. V updates the root using a_8 and only the proof
3. P updates the root

Update



Deletion:

1. P sends proof for a_8 , V validates it
2. V updates the root using only the proof
3. P updates the root

Complexity

- Local storage: $O(1)$
- Server time: $O(\log n)$ with $O(n)$ storage
- Communication: $O(\log n)$
- Verification time: $O(\log n)$
- **Update: $O(\log n)$**